

Doc. No.: P0991R0

Date: 2018-03-12

Reply to: Detlef Vollmann, dv@vollmann.ch

Audience: SG1

Comparison of Stackful Coroutine Proposals

This is a try to understand the differences between the various (stackful) coroutine proposals. It may be useful to others as well.

Terminology

While the distinction between *symmetric* and *asymmetric* seems to be established in literature, it's confusing to me. So I use *switch_to* for what is otherwise called *symmetric* and *suspend/resume* for what others call *asymmetric*.

Blocking

For coroutines *switch_to* and *suspend* are blocking calls. *switch_to* is inherently boost blocking, *suspend* not necessarily.

P0073R2 is the only one that proposes an explicit blocking interface.

Scheduling

All except P0073R2 assume direct switching without scheduler.

Characteristics

For a simplified comparison, some base characteristics were chosen.

compiler vs. library

While it's understood that any coroutine proposal will contain some implementation that's not possible in pure C++ (involving assembler code or compiler intrinsics), some proposals require pretty heavy compiler support.

explicit context

Some proposals require explicit passing/storing of a context object (that's returned by *switch_to/resume*). In other proposals it's just the state of the (stable) coroutine object.

global state

Some proposals require implicit global/thread_local state.

stack allocation/organization control

For stackful coroutines it's sometimes important how the (side) stack is organized and allocated.

switch_to/suspend to main

Allows to switch_to any context, even if not created as coroutine (especially main() or top-level thread functions).

direct data transfer

Allows the switch_to/suspend/resume to transfer data directly.

switch_to

As the switch_to approach is generally more expressive and suspend/resume can generally be built on top of switch_to with no nearly no overhead all but N3708 support switch_to, so it's not a separate coloumn in the table below.

suspend/resume

Some proposals support suspend/resume in addition to switch_to.

syntax

Most proposals provide a pure library interface, but some use a special syntax or keywords.

current()

Some proposals provide a function to get a handle to the current coroutine.

Proposals included in this comparison

- 2013-09/N3708: A proposal to add coroutines to the C++ standard library (Oliver, Nat)
- 2014-05/N3985: A proposal to add coroutines to the C++ standard library (R1) (Oliver, Nat)
- 2015-04/N4397: A low-level API for stackful coroutines, (Oliver)
- 2015-04/N4398: A unified syntax for stackless and stackful coroutines, (Oliver)
- 2015-09/P0099r0: A low-level API for stackful context switching (Nat, Oliver)
- 2016-06/P0073r2: On unifying the coroutines and resumable functions proposals (Torvald)
- 2016-10/P0099r1: A low-level API for stackful context switching (Nat, Oliver)
- 2017-02/p0534r0: call/cc (Oliver)
- 2017-06/p0534r1: call/cc (Oliver)
- 2017-07/p0534r2: call/cc (Oliver)
- 2017-10/p0534r3: call/cc (Oliver)
- 2018-02/p0876r0: fibers without scheduler (Oliver)

Comparison

Paper	Compiler	Explicit Context	Global State	Stack Alloc Control	suspend / resume	switch_to main	Direct Data Transfer	Syntax	current()
N3708	no	no	yes	no	yes	yes	yes	no	no
N3985	no	no	yes	no	yes	yes	yes	no	no
N4397	no	no	yes	yes	no	yes	no	yes	yes

Paper	Compiler	Explicit Context	Global State	Stack Alloc Control	suspend / resume	switch_to main	Direct Data Transfer	Syntax	current()
N4398	yes	no	yes	yes	no	yes	no	yes	yes
P0099R0	no	no	yes	yes	no	yes	no	no	yes
P0073R3	yes	no	?	no	yes	?	no	yes	?
P0099R1	no	yes	no	yes	no	yes	yes	no	no
P0534R0-2	no	yes	no	yes	no	yes	yes	no	no
P0534R3	no	yes	no	yes	no	yes	no	no	no
P0876R0	no	yes	no	yes	no	yes	no	no	no

Remarks to specific papers

N3708/N3985

No direct suspend/resume/switch_to API, but more generator targeted `pull_type` and `push_type` with direct data transfers.

N4397/N4398

Special syntax for resumable lambdas. N4398 allows the compiler to decide between stackless and stackfull.

P0099r0

Conceptually like N4397, but pure library syntax.

P0073r2

Mostly (heavy) compiler techniques for unification and common syntax for stackless and stackful coroutines.

- not much on concrete API
- suspend model assumes scheduler
 - `block()` as hint which coroutine to resume
- switch_to semantics via blocking interface
- `class event` as context object
- low-level as it provides mechanisms for higher-level coroutine executors
- high-level as it hides the execution context and assumes scheduler

P0099r1

Pretty different from P0099R0 to avoid global state.

P0534r0 to r2

Based on P0099R1, no major differences

P0534r3

Update of P0534R2 without direct data transfer.

P0876r0

Essentially a rename of P0534R3.

Assumptions

1. switch_to interface is required
2. no scheduler

Questions

1. Are unified proposals dead?
2. Is suspend/resume required?
3. Is global state acceptable?
 - Is for avoiding global state more undefined behaviour acceptable?
 - Is switch_to main required?
 - Is a more complicated interface (w/o direct mapping to executors) ok?
4. Is there a mechanism to help the compiler to inline a context switch?
5. Should a proposal include a blocking interface?
6. Is control of stack allocation and organization required?
7. Does anybody want direct data transfer?